DSC 40B - Discussion 02

Problem 1.

Are the following asymptotic bounds true?

1. $n \log n = O(n^2)$
2. $\left(\frac{1}{n}\right)^3 = O(\frac{1}{n})^4$
3. $n^{\frac{1}{2}} = \Omega(n^{\frac{1}{3}})$
4. $e^n = O(n!)$
5. $\log n = \Omega(\sqrt{n})$

Solution:

1. $n \log n = O(n^2)$: True. $\frac{n \log n}{n^2} = \frac{\log n}{n} \to 0$ as $n \to \infty$, so you can use the criterion from slides 6 of lecture 3.
2. $\left(\frac{1}{n}\right)^3 = O\left(\frac{1}{n}\right)^4$: False $\frac{\left(\frac{1}{n}\right)^3}{O\left(\frac{1}{n}\right)^4} = n \to \infty$ as $n \to \infty$
3. $n^{\frac{1}{2}} = \Omega(n^{\frac{1}{3}})$: True: $\frac{n^{\frac{1}{2}}}{n^{\frac{1}{3}}} = n^{\frac{1}{6}} \to \infty$ as $n \to \infty$
4. $e^n = O(n!)$ True. An easy way to see it is to write the following

$$\begin{aligned}\frac{e^n}{n!} &= \frac{e \times e \times e \ldots \times e}{1 \times 2 \times 3 \ldots \times n} \\ &= \frac{e}{1} \times \frac{e}{2} \times \frac{e}{3} \times \underbrace{\frac{e}{4}}_{<1} \times \underbrace{\frac{e}{5}}_{<1} \ldots \times \underbrace{\frac{e}{n}}_{<1} \text{ (remember that } e < 4) \\ &\leq \frac{e}{1} \times \frac{e}{2} \times \frac{e}{3} = \frac{e^3}{6} \text{ which is a constant}\end{aligned}$$

5. $\log n = \Omega(\sqrt{n})$: False: $\frac{\log n}{\sqrt{n}} \to 0$ as $n \to \infty$

Problem 2.

a) Let $f(n) = 12log_2(3^{n^2-2n} + 2^{logn} - 10n^2 - log_3 n)$. Which of the following asymptotic bounds on f is true?

1. $f(n) = \Omega(1)$
2. $f(n) = O(1)$
3. $f(n) = O(\log n)$
4. $f(n) = \Omega(\log n)$
5. $f(n) = \Theta(\log n)$
6. $f(n) = O(n)$
7. $f(n) = O(e^n)$
8. $f(n) = \Theta(n)$
9. $f(n) = \Theta(n^2)$

Solution: $f(n) = \Theta(n^2)$
it follows from that that:

1. $f(n) = \Omega(1)$ (ie f is bounded below by a constant after some time)
2. $f(n) \neq O(1)$ (ie f is not bounded)
3. $f(n) \neq O(\log n)$
4. $f(n) = \Omega(\log n)$
5. $f(n) \neq \Theta(\log n)$ (because of 3)
6. $f(n) \neq O(n)$
7. $f(n) = O(e^n)$
8. $f(n) \neq \Theta(n)$ (because of 6)
9. $f(n) = \Theta(n^2)$

This question is meant to develop some intuition for finding asymptotic bounds on tricky functions, so we won't write up the formal proof (although it is possible, just difficult algebraically.) We want to start off by finding a simpler expression which is approximately equal to the expression inside the log for large values of n:

$3^{n^2-2n} + 2^{logn} - 10n^2 - log_3 n \approx 3^{n^2-2n}$ as n grows to infinity, since 3^{n^2-2n} is the highest order term. Furthermore, since n^2 grows much faster than $-2n$, $3^{n^2-2n} \approx 3^{n^2}$. So, we have $f(n) \approx 12log_2(3^{n^2})$.

Applying log rules, we can bring down the exponent to get $12log_2(3^{n^2}) = n^2 \cdot 12log_2(3)$. Since $12log_2(3)$ is a constant with respect to n, we now know that this function is approximately $\theta(n^2)$.

b) What is the best case time complexity of the following function?

```
def foo(arr):
''' arr is a sorted array of size n'''
        i = 0
        j = len(arr) - 1

        while i < j:
                current_sum = arr[i] + arr[j]

                if current_sum == 5:
                        return sum(arr)
                elif current_sum < 5:
                        i += 1
                else:
                        j -= 1

        return False
```

Solution: $\Theta(n)$
There are 2 cases to consider:

1. We reach
   `return sum(arr)`
   at some point in the loop. In that case, the best case time complexity is if we reach it on the first iteration, which gives us $\Theta(n)$.
2. We never reach
   `return sum(arr)`

. In this case the while loop runs approximately $n/2$ times, so the complexity is still $\Theta(n)$.

Both those cases have the same complexity, so the best-case complexity is $\Theta(n)$

Please also review problems 3 and 4 from discussion 1. They will help towards homework 2. In discussion on Friday, we will go over some of the problems from Homework 1 and 2.